

Hands-on HPC Python: Speeding Up Your Code

Vincent Drach

June 2026

School of Computing, Electronics and Mathematics
vincent.drach@plymouth.ac.uk

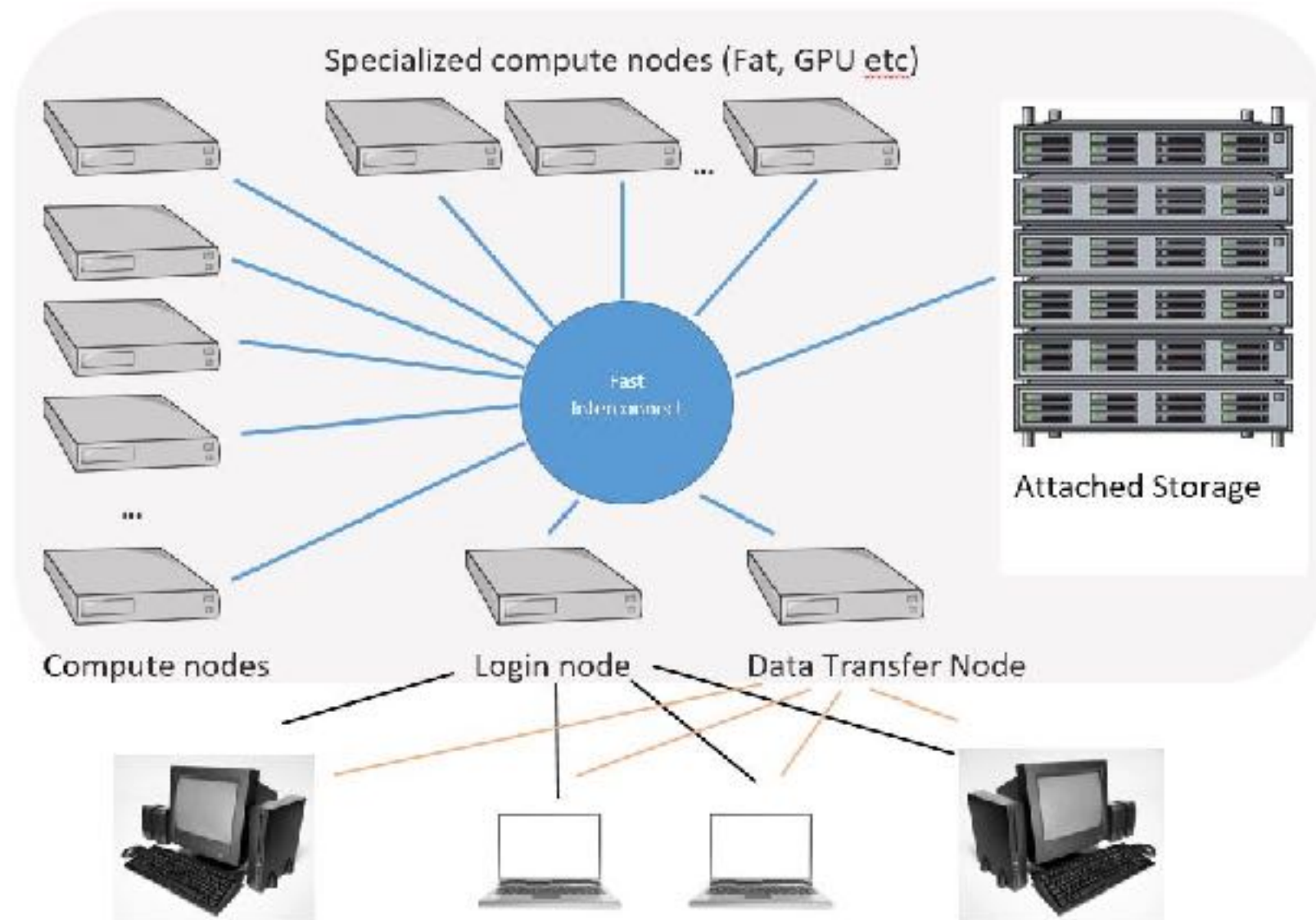
Today's goals

- Quick tour of the facility
- A bit of theory: computing Pi - a code Kata ?
- Access lovelace and configure your working environment
- First implementations - definition of a benchmark
- Lessons : loading a function, numpy and vectorisation
- Just-in-time compilation : numba
- A bit of profiling
- Parallel computing : embarrassingly parallel tasks, hidden parallelism, multithreading,...
- GPUs : how does a GPU work, using cupy, jax and numba.
- Summary

Quick tour of the facility

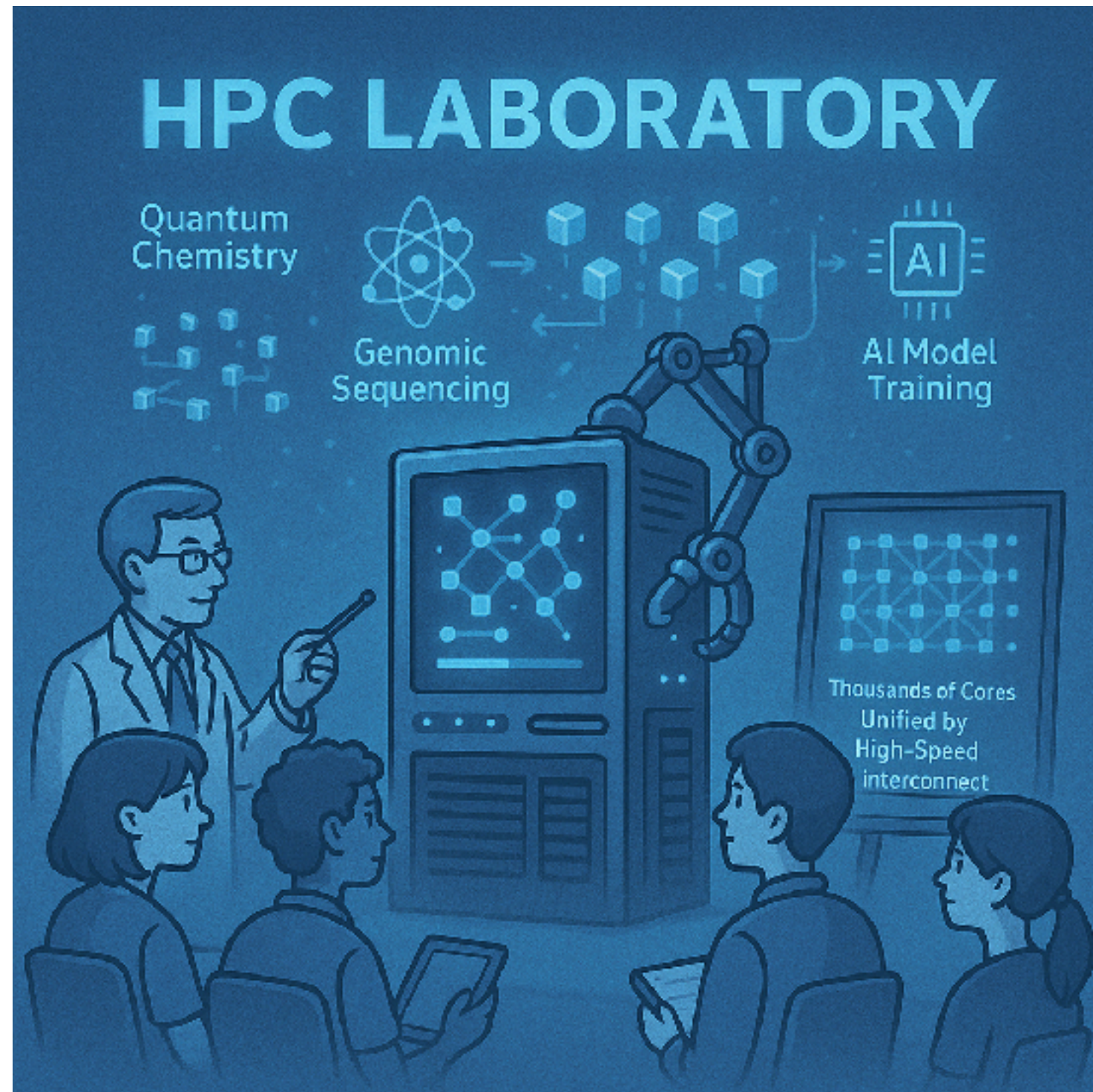
High Performance Computing

- High Performance Computing cluster : collection of servers, connected by a fast interconnect
- Type of resources:
 - ◆ Multi-core CPU
 - ◆ GPUs

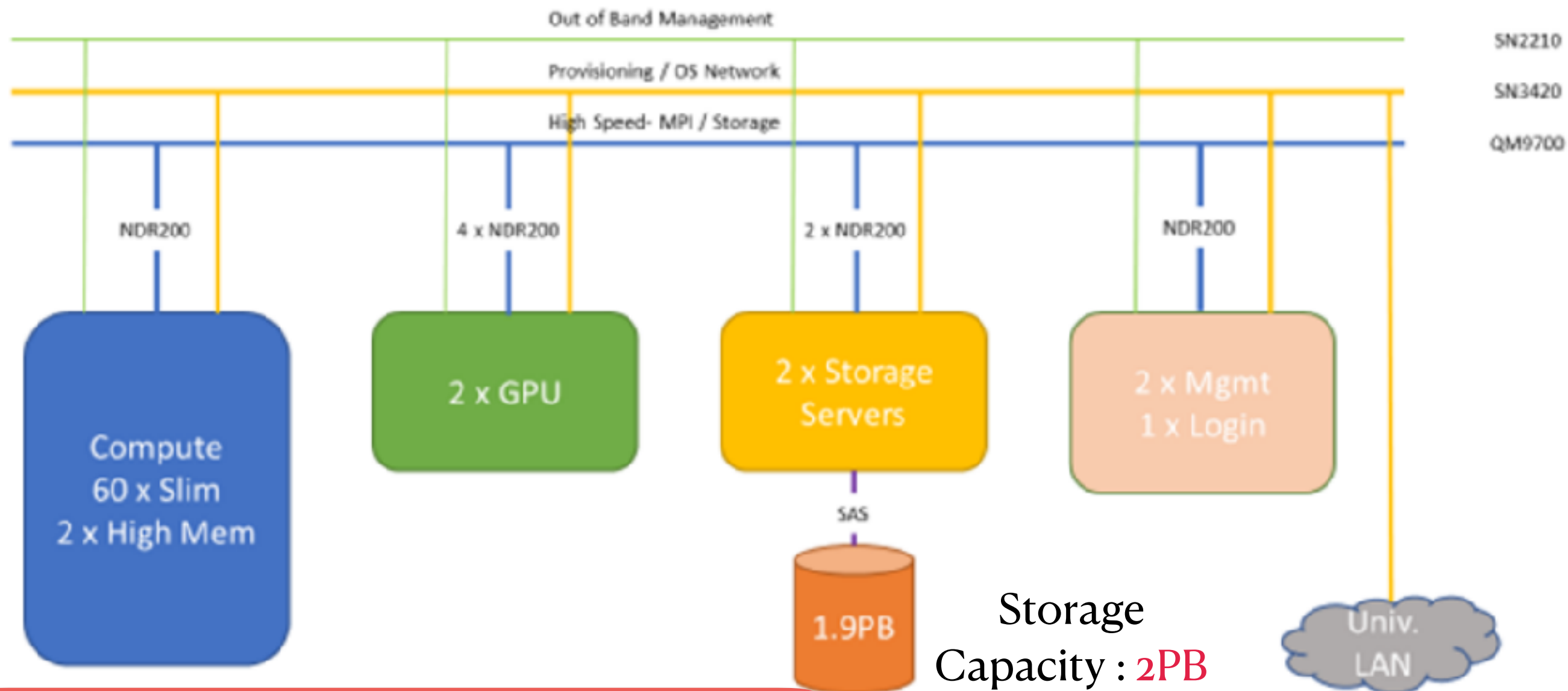


High Performance Computing

- Depending on your scientific problem you might
 - ✦ not need to use multiple nodes
 - ✦ use a software that orchestrates the communication between the nodes for you.
 - ✦ Need significant developments
- High Performance Computing environment requires
 - ✦ To analyse the computing problem that you are trying to solve
 - ✦ To accept that part of the research is to learn how to use the hardware
 - ✦ To do some experimentation



Lovelace system (March 2025)



60x CPU nodes :

AMD EPYC 64 cores per nodes
384 GB of RAM

2x High Mem nodes :

AMD EPYC 64 cores per nodes
1,536GB of RAM

GPU Server #1

8 x L40S Nvidia GPUs
1,536GB of RAM

GPU Server #2

4 x H100 SXM5 80G Nvidia GPUs
Fully meshed NVlink
1,536GB of RAM

NVIDIA QM9700 NDR Managed
Infiniband Switch.

400Gbit/s

Computing Nodes

The H100 was the most powerful GPU chip on the market up a a few month ago !

Common features of HPC machines

- A Linux Machine that can be access remotely
- **Module Manager**: A way to load **your** tools
- **Job scheduler**: that dispatches and orchestrates jobs on the computing nodes - your jobs might not run immediately
- You can/must **fine-tune your environment** to your need
- You can access the machine using web-browser interface without any linux skills.

Today we will focus on the last 2 points.

Login in Lovelace (Linux)

- On University machine: download mobaxterm
- Alternative for windows users : WSL (Window Subsystem for Linux)

See Useful Link section on <https://docs.lovelace.plymouth.ac.uk/>

- Start MobaXterm
- Type :

ssh username@lovelace.plymouth.ac.uk

- Enter your password
- You are now connected on the head node of Lovelace.

Accessing Lovelace

- Look for an email by “hpc-applications@plymouth.ac.uk” and follow the instructions to set a password.
- Login : <https://jupyter.lovelace.plymouth.ac.uk>
- Choose the workload profile: **General Purpose**
- Choose the ressource profile : **Large (64 CPU cores, 365 GiB Memory)**
- **Today** : we ask you to book a large node. This is not recommended in general for debugging/developpment.

Environment configuration - linking with Jupyterhub

- Imagine you want to configure an environment in jupyterhub containing a number of centrally available modules and some additional one from pip
- Write a *my_HPC_env.toml* file containing

```
name = "hpc_pi_test"
human_name = "HPC Python Test"
pip_packages = "threadpoolctl line_profiler memory_profiler snakeviz ipykernel"
modules = [
    "spack-modules/293",
    "python-science/py-numpy-py3.11/1.26.4-k5adanv",
    "compilers/llvm/14.0.6-xpvm6vv",
    "python-science/py-numba-py3.11/0.59.1-ijufvg",
    "py-virtualenv-py3.11/20.26.6-xqjpfog",
    "python-science/py-matplotlib-py3.11/3.10.6-ogsaoll",
    "deeplearning/py-jax-py3.11",
]
```

Then run the following line in your terminal (login node). Of course make sure the toml is available.

```
/scratch/scripts/setup_jupyter my_HPC_env.toml
```

You can change name, human_name, pip_packages and modules to suit your needs. In this example, "HPC Python Test" will then be one of the kernels available in jupyterhub.

Computing Pi

The problem we will solve

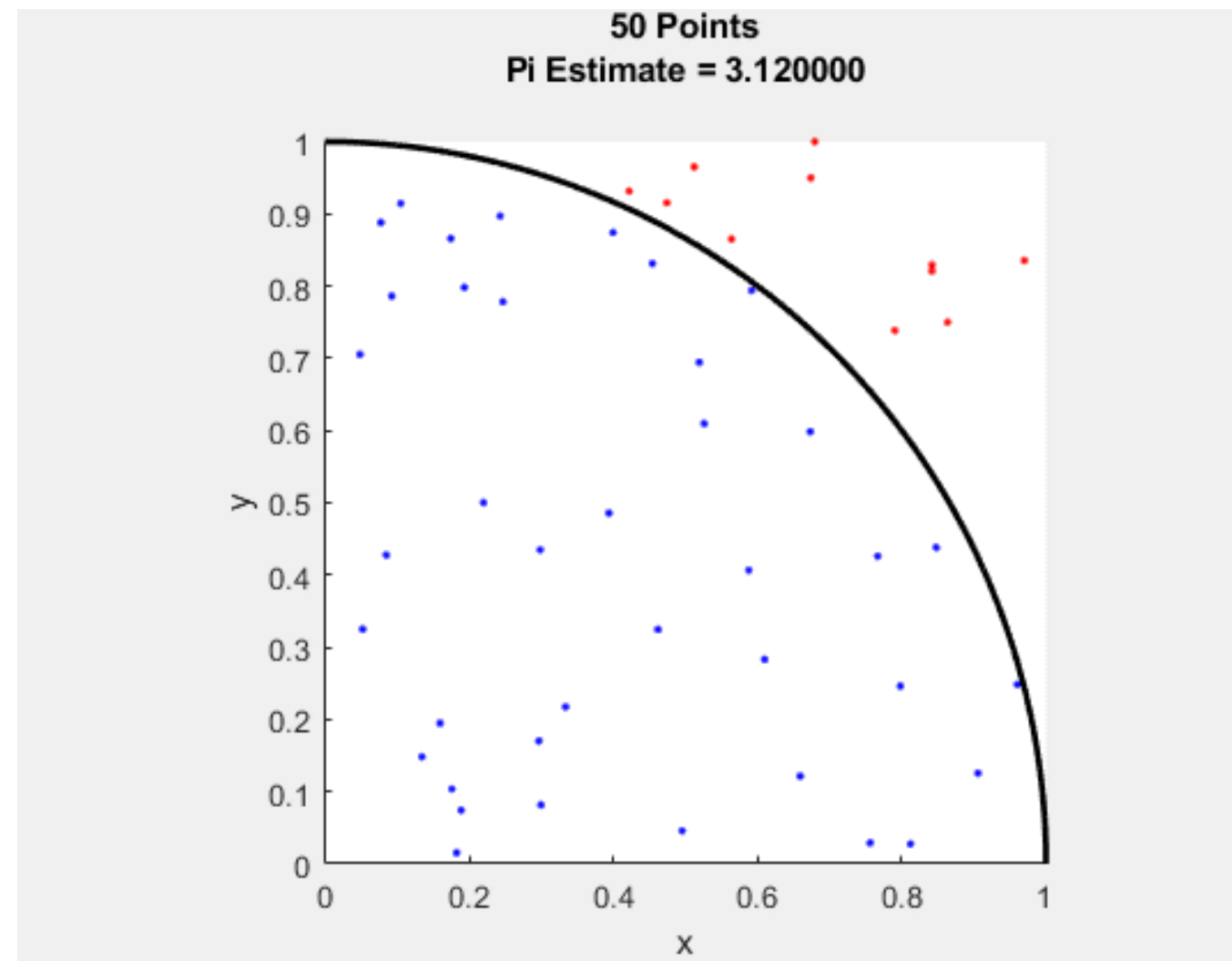
- An approximation of π using a Monte-Carlo statistical approach is given using a random set of m points :

$$X_m = (x, y)_m \in [0, 1]^2$$

- The probability that a point lies inside a quarter of the unit disk is given by :

$$P = \frac{\pi}{4} = \frac{\#(x^2 + y^2 \leq 1)}{m}.$$

- To approximate π , we produce m points and count how many lie inside a quarter of the unit disk



(see Matlab tutorial 2025 - special thanks to Mike Cr)

Note: this is a Monte-Carlo method that is used in many area...

First attempt

```
import random
import time
import math
```

```
def MCiter():
    x = random.random()
    y = random.random()
    count=0
    if x**2+y**2 < 1:
        count=1
    return count
```

```
def compute_pi_baseline_v0(Niter):

    count=0
    for i in range(Niter):
        count+=MCiter() # function call inside loop

    piEst=4*count/Niter

    return piEst
```

Can you be slower than me ?

How to time it ?

```
Niter = 5_000_000

start = time.perf_counter()
piEst=compute_pi_baseline_v0(Niter)
end = time.perf_counter()
timeTaken = end - start

print(f"Time taken for v0 {Niter} : {timeTaken} s")
print(f"Estimate for pi is {piEst} after {timeTaken} seconds")
print(f"Absolute error is {abs(piEst-math.pi)}")
print(f"speed = {Niter/timeTaken/1e6} million samples per
second\n",)
```

- I get (H100 node - not using the GPU !):

```
Time taken for v0 5000000 : 0.7664118018001318 s
Estimate for pi is 3.141776 after 0.7664118018001318 seconds
Absolute error is 0.00018334641020700815
speed = 6.523907889017504 million samples per second
```

Stochastic noise

- You can repeat the experience. You will see that the time and the speed does not depend on the number of samples.
- Note that the absolute error = $\text{abs}(\text{estimate} - \pi)$ fluctuates quite a bit ! This is inherent to our algorithm. It can be shown that for Niter large enough :

$$\text{Abs. Error} \sim \frac{C}{\sqrt{N_{\text{iter}}}}$$

- So you multiply Niter by 100 , the error is decreased by a factor 10.
- Use instead

`%timeit compute_pi_baseline_v0(Niter)` [Reports the average execution time per run of the code.]

- I get : 774 ms $\pm$ 10.9 ms per loop (mean $\pm$ std. dev. of 7 runs, 1 loop each)
- Note that we can safely assume that :

$$\text{Time Taken} \sim k N_{\text{iter}}$$

How many samples are needed ?

- Imagine we would need **7 decimal places**. How many samples would be needed ?
- We could repeat the experiment many times for a given Niter, estimate the mean of the absolute error, and determine how many more samples we would need to get to an accuracy of 1e-7.

$$N = N_0 \left(\frac{\text{err}_0}{\text{err}} \right)^2$$

- Example :

For $N_0 = 5_000_000$; I get $\text{err}_0 = 0.0006 \sim 6e-4$ (mean of the absolute difference)

So I would a factor 6000 in the error = 36 million in statistics

-> I would need Niter = $1.8e14$

~372 days!

How to access Lovelace

- Look for an email by “hpc-applications@plymouth.ac.uk” and follow the instructions to set a password.
- Login : <https://jupyter.lovelace.plymouth.ac.uk>
- Choose the workload profile: **General Purpose**
- Choose the ressource profile : **Large (64 CPU cores, 365 GiB Memory)**
- **Today** : we ask you to book a large node. This is not recommended in general for debugging/developpment.

Second attempt

```
def compute_pi_baseline_v1(Niter):  
  
    count=0  
    for i in range(Niter):  
        x = random.random()  
        y = random.random()  
        if x**2+y**2 < 1:  
            count+=1  
  
    piEst=4*count/Niter  
  
    return piEst
```

Third attempt

```
def compute_pi_baseline_v2(Niter):  
    rng = random.random  
  
    count=0  
    for i in range(Niter):  
        x = rng()  
        y = rng()  
        if x**2+y**2 < 1:  
            count+=1  
  
    piEst=4*count/Niter  
  
    return piEst
```

Fourth attempt

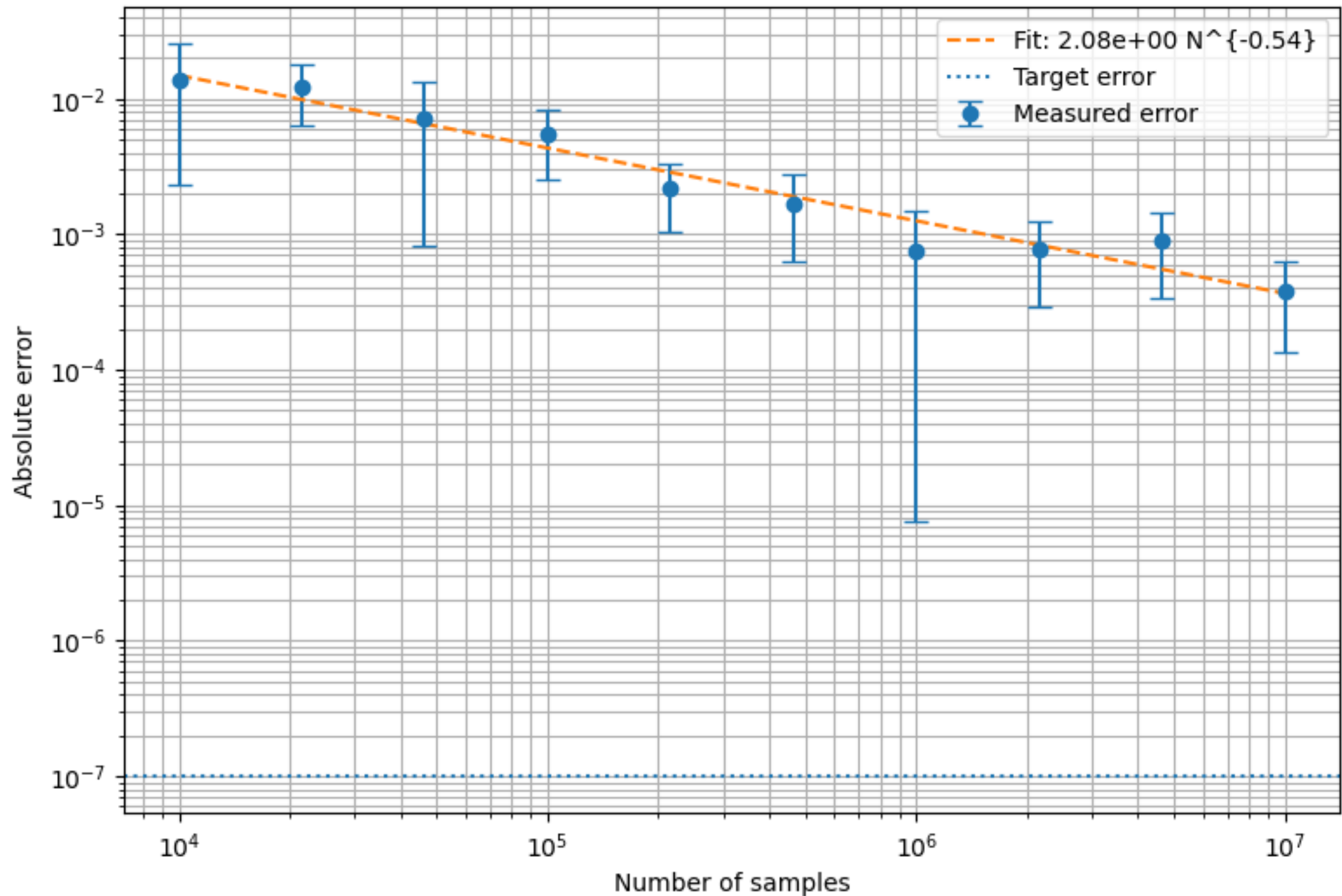
```
def compute_pi_baseline_v3(Niter):  
  
    rng = random.random  
  
    count=0  
    for i in range(Niter):  
        x = rng()  
        y = rng()  
        if x*x+y*y < 1: # x*x is faster than x**2  
            count+=1  
  
    piEst=4*count/Niter  
  
    return piEsti
```


Summary

Function	Speed (Million samples per second)	Time to 10 ⁻⁷ precision (Days)
compute_pi_baseline_v0	6.5	~ 372
compute_pi_baseline_v1	7.78	~ 213
compute_pi_baseline_v2	8.39	~ 193
compute_pi_baseline_v3	10.04	~ 167

We could do better (example for v1)

Monte Carlo π Error Scaling



Using that model I get 81 days

Summary

Function	Speed (Million samples per second)	Time to 10^{-7} precision [naive scaling]	Time to 10^{-7} precision [fit and extrapolation]
compute_pi_baseline_v0	6.5	~ 372 days	
compute_pi_baseline_v1	7.78	~ 213 days	~ 81 days
compute_pi_baseline_v2	8.39	~ 193 days	~ 43 days
compute_pi_baseline_v3	10.04	~ 167 days	~ 26 days

How can we improve further

- Many routes:
 - ♦ Algorithm (e.g random number)
 - ♦ Profiling
 - ♦ Vectorisation
 - ♦ Just in time compilation
 - ♦ Parallel CPU
 - ♦ GPU computing

Finding bottleneck: profiling

Performance profiling

Why Profile?

Before optimising code, we need to identify where time and memory are being spent.

Common mistakes:

- Optimising code that is not performance-critical.
- Assuming the bottleneck without measuring.
- Making code more complex without improving performance.

Golden Rule

Measure first. Optimise second.

Finding bottlenecks

- Function-Level Profiling

```
%prun -s cumulative
```

- Measures time spent inside functions.

Example:

```
%prun -s cumulative compute_pi_baseline_v3(10_000_000)
```

- Useful for:

- Identifying expensive functions.
- Understanding the call hierarchy.
- Locating performance bottlenecks

Finding bottlenecks

- **Line-by-Line Profiling:** `lprun -f`
- Measures execution time for individual line.

Example: `%lprun -f cumulative compute_pi_baseline_v3(10_000_000)`

- Useful for:
 - Pinpointing slow lines of code.
 - Understanding where time is spent inside a function.

Memory profiling: `%memit`

- Measures peak memory usage.

Example: `%memit cumulative compute_pi_baseline_v3(10_000_000)`

- Useful for:
 - Detecting large memory allocations.
 - Understanding memory bottleneck.

Examples

Collapse Output

```
%lprun -f compute_pi_baseline_v1 compute_pi_baseline_v1(1_000_000)
```

2000004 function calls in 0.361 seconds

Ordered by: cumulative time

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	0.361	0.361	{built-in method builtins.exec}
1	0.000	0.000	0.361	0.361	<string>:1(<module>)
1	0.281	0.281	0.361	0.361	2720131820.py:25(compute_pi_baseline_v1)
2000000	0.080	0.000	0.080	0.000	{method 'random' of '_random.Random' objects}
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.Profiler' objects}

```
%lprun -f compute_pi_baseline_v1 compute_pi_baseline_v1(1_000_000)
```

Timer unit: 1e-09 s

Total time: 0.621704 s

File: /tmp/ipykernel_1349414/2720131820.py

Function: compute_pi_baseline_v1 at line 25

Line #	Hits	Time	Per Hit	% Time	Line Contents
25					def compute_pi_baseline_v1(Niter):
26					
27					
28	1	411.0	411.0	0.0	count=0
29	1000001	88432207.0	88.4	14.2	for i in range(Niter):
30	1000000	135296833.0	135.3	21.8	x = random.random()
31	1000000	120998601.0	121.0	19.5	y = random.random()
32	1000000	179000522.0	179.0	28.8	if x**2+y**2 < 1:
33	785344	97974221.0	124.8	15.8	count+=1
34					
35	1	1231.0	1231.0	0.0	piEst=4*count/Niter
36					
37	1	90.0	90.0	0.0	return piEst

Examples

Collapse Output

```
%lprun -f compute_pi_baseline_v1 compute_pi_baseline_v1(1_000_000)
```

2000004 function calls in 0.361 seconds

Ordered by: cumulative time

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	0.361	0.361	{built-in method builtins.exec}
1	0.000	0.000	0.361	0.361	<string>:1(<module>)
1	0.281	0.281	0.361	0.361	2720131820.py:25(compute_pi_baseline_v1)
2000000	0.080	0.000	0.080	0.000	{method 'random' of '_random.Random' objects}
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.Profiler' objects}

```
%lprun -f compute_pi_baseline_v1 compute_pi_baseline_v1(1_000_000)
```

Timer unit: 1e-09 s

Total time: 0.621704 s

File: /tmp/ipykernel_1349414/2720131820.py

Function: compute_pi_baseline_v1 at line 25

Line #	Hits	Time	Per Hit	% Time	Line Contents
25					def compute_pi_baseline_v1(Niter):
26					
27					
28	1	411.0	411.0	0.0	count=0
29	1000001	88432207.0	88.4	14.2	for i in range(Niter):
30	1000000	135296833.0	135.3	21.8	x = random.random()
31	1000000	120998601.0	121.0	19.5	y = random.random()
32	1000000	179000522.0	179.0	28.8	if x**2+y**2 < 1:
33	785344	97974221.0	124.8	15.8	count+=1
34					
35	1	1231.0	1231.0	0.0	piEst=4*count/Niter
36					
37	1	90.0	90.0	0.0	return piEst

Numpy: vectorisation

Vectorisation :



- Vectorisation means performing operations on **entire arrays at once** instead of using explicit Python loops.
- Typically using **numpy** or **pandas**
- Convince yourself it is the case, compare for instance (for large arrays!):

```
result = []  
for x in arr:  
    result.append(x * 2)
```

```
# vectorisation :  
result = arr * 2
```



Vectorisation :

```
def compute_pi_numpy(Niter) :  
  
    x = np.random.random(Niter)  
    y = np.random.random(Niter)  
  
    inside = (x*x + y*y) < 1.0  
  
    count = np.sum(inside)  
  
    piEst = 4.0 * count / Niter  
  
    return piEst
```

◦ Results N = 5_000_000 ;

Time taken for v0 5000000 : 0.06121007585898042 s
Estimate for pi is 3.14191556 after 0.06121007585898042 seconds
Absolute error of the mean is 0.0003229064102070467
Mean of the Absolute error is 0.00045329856408276025
N/N0 20547958.819949232
speed = 81.68589778452997 million samples per second

Estimation time for 1e-07 precision = 14.557200441160859 days; Niter target = 1.0273979409974616x10¹⁴

Summary

Function	Speed (Million samples per second)	Time to 10^{-7} precision [naive scaling]	Time to 10^{-7} precision [fit and extrapolation]
compute_pi_baseline_v0	6.5	~ 372 days	
compute_pi_baseline_v1	7.78	~ 213 days	~ 81 days
compute_pi_baseline_v2	8.39	~ 193 days	~ 43 days
compute_pi_baseline_v3	10.04	~ 167 days	~ 26 days
compute_pi_numpy	81.68	~14 days	

What is vectorisation?

```
def compute_pi_numpy(Niter):  
  
    x = np.random.random(Niter)  
    y = np.random.random(Niter)  
  
    inside = (x*x + y*y) < 1.0  
  
    count = np.sum(inside)  
  
    piEst = 4.0 * count / Niter  
  
    return piEst
```

◦ Vectorised Monte Carlo π Estimation

- ◆ Generates **Niter random (x, y) points** simultaneously using NumPy arrays.
- ◆ Uses vectorised arithmetic to compute x^2+y^2 for all points at once.
- ◆ Creates a **Boolean mask** (`inside`) identifying points inside the unit circle.
- ◆ Counts points inside the circle efficiently with `np.sum( )`.

Why vectorised estimation is faster

```
def compute_pi_numpy(Niter):  
  
    x = np.random.random(Niter)  
    y = np.random.random(Niter)  
  
    inside = (x*x + y*y) < 1.0  
  
    count = np.sum(inside)  
  
    piEst = 4.0 * count / Niter  
  
    return piEst
```

Why is it faster ?

- Operations are executed in optimized **C code** behind NumPy.
- Reduces slow Python loop overhead.
- Better memory and CPU utilization.

Key Takeaway

Vectorisation makes NumPy code **shorter, cleaner, and significantly faster** by applying operations to whole arrays simultaneously.

Numba: Just in time compilation

Numba Decorators: Telling Numba What to Optimise

◦ What is a Decorator?

- A **decorator** is a special line placed above a function definition.
- It modifies or extends the function's behaviour.
- In Numba, decorators tell Python to **compile the function for faster execution**.

Example:

The `@njit` line tells Numba:

```
from numba import njit, prange, cuda
```

```
@njit
```

```
def compute_pi_numba_njit(Niter):
```

```
    count = 0
```

```
    for i in range(Niter):
```

```
        x = np.random.random()
```

```
        y = np.random.random()
```

```
        if x*x + y*y < 1.0:
```

```
            count += 1
```

```
    piEst = 4.0 * count / Niter
```

```
    return piEst
```

Numba & Just-In-Time (JIT) Compilation

Results : N=5_000_000

Time taken for v0 5000000 : 0.02559358114376664 s
Estimate for pi is 3.1415332 after 0.02559358114376664 seconds
Absolute error of the mean is 5.9453589793090345e-05
Mean of the Absolute error is 0.00047672000000003044
N/N0 22726195.8400029
speed = 195.3614842687913 million samples per second

Estimation time for 1e-07 precision = 6.731999274539887 days; Niter target = 1.1363097920001453x10¹⁴

Function	Speed (Million samples	Time to 10 ⁻⁷ precision	Time to 10 ⁻⁷ precision
compute_pi_baseline_v0	6.5	~ 372 days	
compute_pi_baseline_v1	7.78	~ 213 days	~ 81 days
compute_pi_baseline_v2	8.39	~ 193 days	~ 43 days
compute_pi_baseline_v3	10.04	~ 167 days	~ 26 days
compute_pi_numpy	81.68	~14 days	
compute_pi_numba	195.36	~6.7 days	

Controlling numba

Common Numba Options

nopython=True (used by `@njit`)

- Compiles the entire function to machine code.
- No Python interpreter involved during execution.
- Fastest execution mode.

nopython=False

- Falls back to Python for unsupported operations.
- More flexible but generally slower.

parallel=True

- Allows Numba to use multiple CPU cores.
- Can significantly speed up large loop-based calculations.
- Often combined with `prange()`.

Key Takeaway

A **decorator** is simply an instruction placed above a function. Numba decorators tell Python to **compile numerical code at runtime**, often giving large performance improvements with minimal code changes.

Numba parallel

Example:

The `@njit(parallel=True)` line tells Numba to parallelise the loop (note the *prange*)

```
@njit(parallel=True)
def compute_pi_numba_parallel(Niter):

    count = 0

    for i in prange(Niter):

        x = np.random.random()
        y = np.random.random()

        if x*x + y*y < 1.0:
            count += 1

    piEst = 4.0 * count / Niter

    return piEst

res = compute_pi_numba_parallel(100) #the function gets compiled.
```

Numba & Just-In-Time (JIT) Compilation

Results : N=5_000_000

Time taken for v0 5000000 : 0.0016494630835950375 s
Estimate for pi is 3.14169284 after 0.0016494630835950375 seconds
Absolute error of the mean is 0.00010018641020703711
Mean of the Absolute error is 0.0004955892820413865
N/N0 24560873.6474297
speed = 3031.289423648331 million samples per second

Estimation time for 1e-07 precision = 0.4688918331282117 days; Niter target = 1.228043682371485x10¹⁴

Function	Speed (Million	Time to 10 ⁻⁷ precision	Time to 10 ⁻⁷ precision
compute pi baseline v0	6.5	~ 372 days	
compute_pi_baseline_v1	7.78	~ 213 days	~ 81 days
compute_pi_baseline_v2	8.39	~ 193 days	~ 43 days
compute_pi_baseline_v3	10.04	~ 167 days	~ 26 days
compute_pi_numpy	81.68	~14 days	
compute_pi_numba	195.36	~6.7 days	
compute_pi_numba_par	3031	~0.46 day	

Conclusion: Numba

- Numba provides a simple way to **accelerate Python code** without rewriting it in C/C++.
- Particularly effective for **loop-heavy numerical computations** where vectorisation is difficult or impractical.
- Uses **Just-In-Time (JIT) compilation** to convert Python functions into optimized machine code at runtime.
- Can deliver substantial performance improvements while keeping code readable and maintainable.
- Integrates naturally with NumPy arrays and scientific Python workflows.

Looking Ahead

So far, we have used Numba to accelerate code on the **CPU**. In a later session, we will explore **Numba + CUDA**, where computations can be offloaded to a **GPU**

Using threads

Threads and the Shared-Memory Model (hardware side)

Example Compute Node

- **64 CPU cores** available for computation.
- All cores can access the node's main memory (RAM).
- Different cores can work on the same dataset.

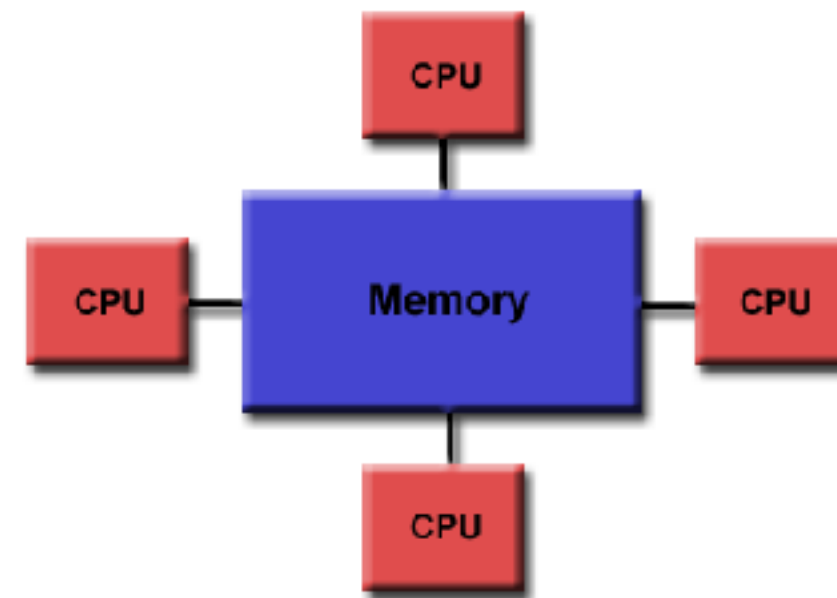
In Practice

- Modern nodes are often divided into **NUMA (Non-Uniform Memory Access) domains**.
- Memory is physically attached to a subset of the CPUs.
- Accessing "local" memory is faster than accessing memory attached to another NUMA domain.

Simplified View

For this lecture, we will treat the node as a **shared-memory machine**:

- All cores can access the same address space.
- Threads communicate through shared memory.



Key Idea

A compute node provides many CPU cores that can operate simultaneously on data stored in a shared memory space.

Threads and the Shared-Memory Model (software side)

What is a Thread?

- A thread is an independent execution path within a process.
- Multiple threads can run concurrently.
- Used to exploit multi-core CPUs.

Shared Memory

- All threads access the same memory space.
- Large arrays do not need to be copied.
- Threads can work on different parts of the same data.

Key Idea

Threads share data and can execute simultaneously on different CPU cores.

Hidden Parallelism in Scientific Python

Parallelism Without Writing Threads

- Many NumPy operations call optimized native libraries.
- These libraries may already use multiple CPU cores.
- Parallelism is often invisible in Python code.

Example: $C = A @ B$ (matrix product of the 2 square matrices A and B)

Behind the scenes: Python -> Numpy -> BLAS -> Multiple threads.

Why It Matters

- Your code may already be parallel.
- Adding more threads is not always beneficial.
- Performance gains often come "for free".

Key Idea

NumPy may already be using multiple cores through BLAS libraries.

Let's try !

Hidden Parallelism in Scientific Python

Parallelism Without Writing Threads

- Many NumPy operations call optimized native libraries.
- These libraries may already use multiple CPU cores.
- Parallelism is often invisible in Python code.

Example: $C = A @ B$ (matrix product of the 2 square matrices A and B)

Behind the scenes: Python -> Numpy -> BLAS -> Multiple threads.

Why It Matters

- Your code may already be parallel.
- Adding more threads is not always beneficial.
- Performance gains often come "for free".

Key Idea

NumPy may already be using multiple cores through BLAS libraries.

Let's try !

Hidden Parallelism in Scientific Python

```
# kernel must be killed to run this.
```

```
import time
import sys
import os
```

```
verbose=True
```

```
# without that line the default chosen by python would be the number of cores actually available in
your jupyter session !
```

```
os.environ["OMP_NUM_THREADS"] = "64"
```

```
nthreads=os.environ['OMP_NUM_THREADS']
```

```
#nthreads = 'unknown'
```

```
import numpy as np
```

```
N = 2000
```

```
# Create two large random matrices
```

```
a = np.random.randn(N, N)
```

```
b = np.random.randn(N)
```

```
Mem = (a.nbytes + b.nbytes)
```

```
# matrix vector mult
```

```
nrep=100
```

```
t1 = time.time()
```

```
for i in range(nrep):
```

```
    b= a@b
```

```
dt = (time.time()-t1)/nrep
```

```
if verbose == True:
```

```
    print(f"Number of threads used: {nthreads}")
```

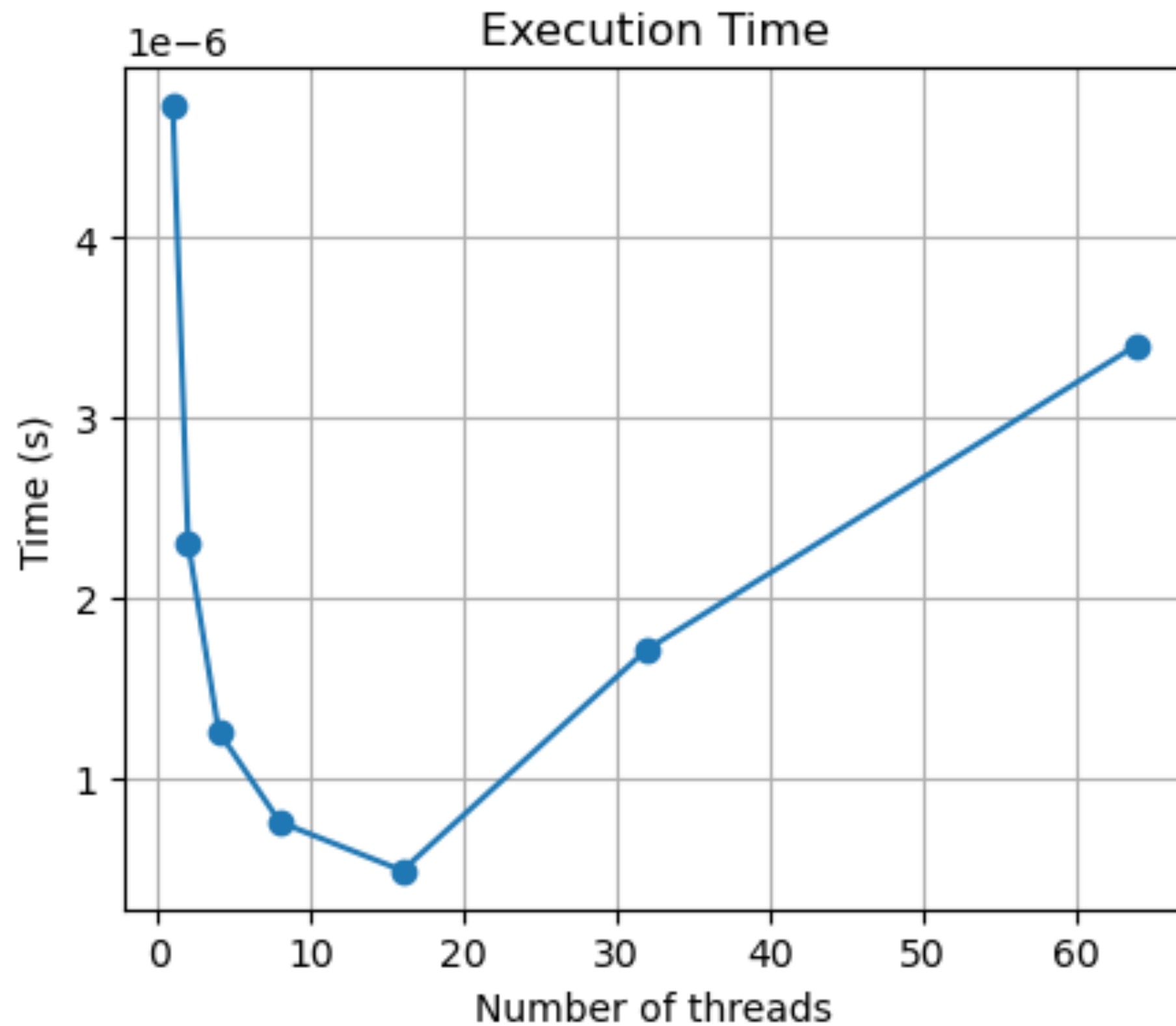
```
    print(f"Memory used {(a.nbytes + b.nbytes)*1e-6}MB")
```

```
    print(f"Time to 1 matrix vector multiplication {dt/nrep} seconds")
```

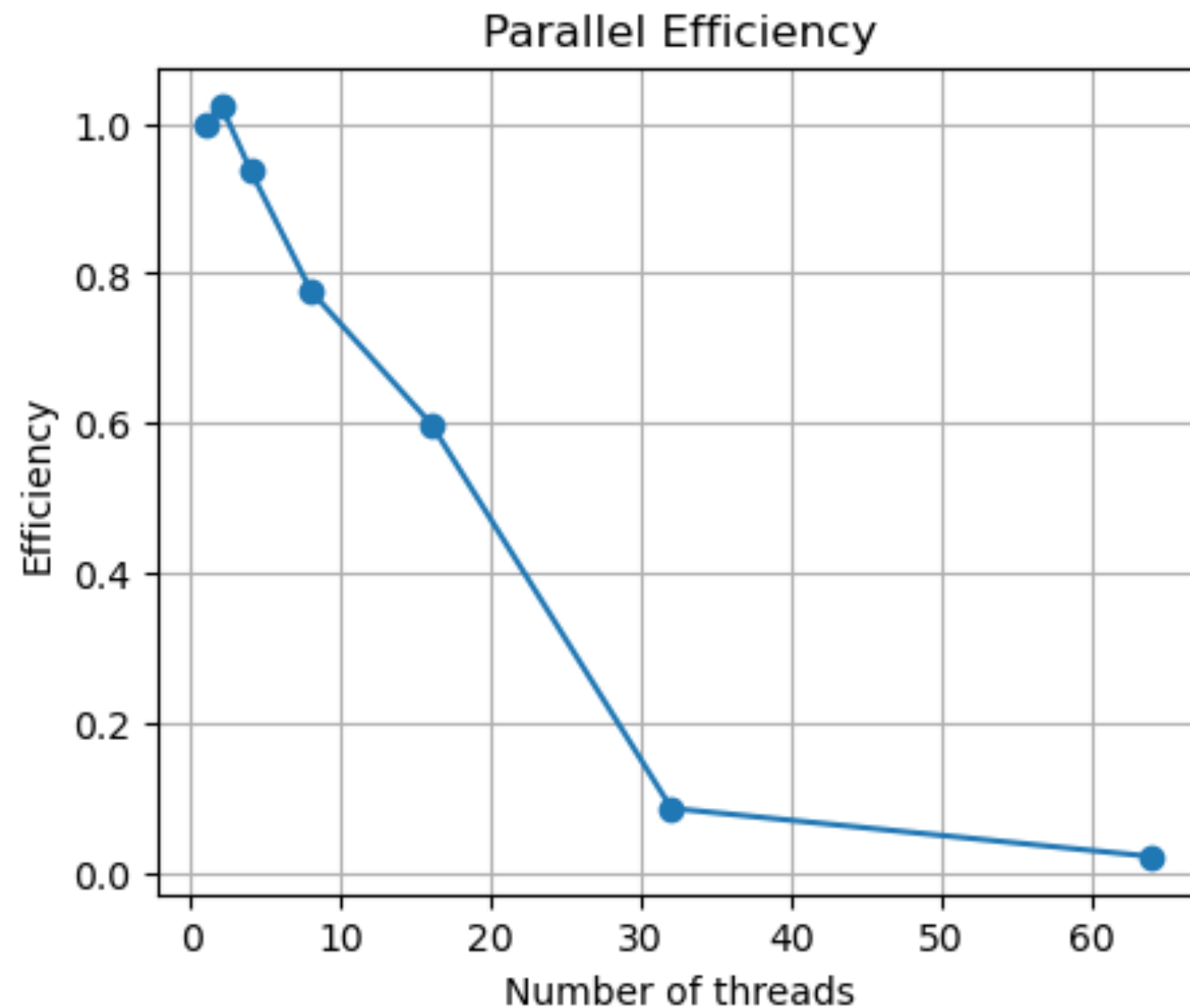
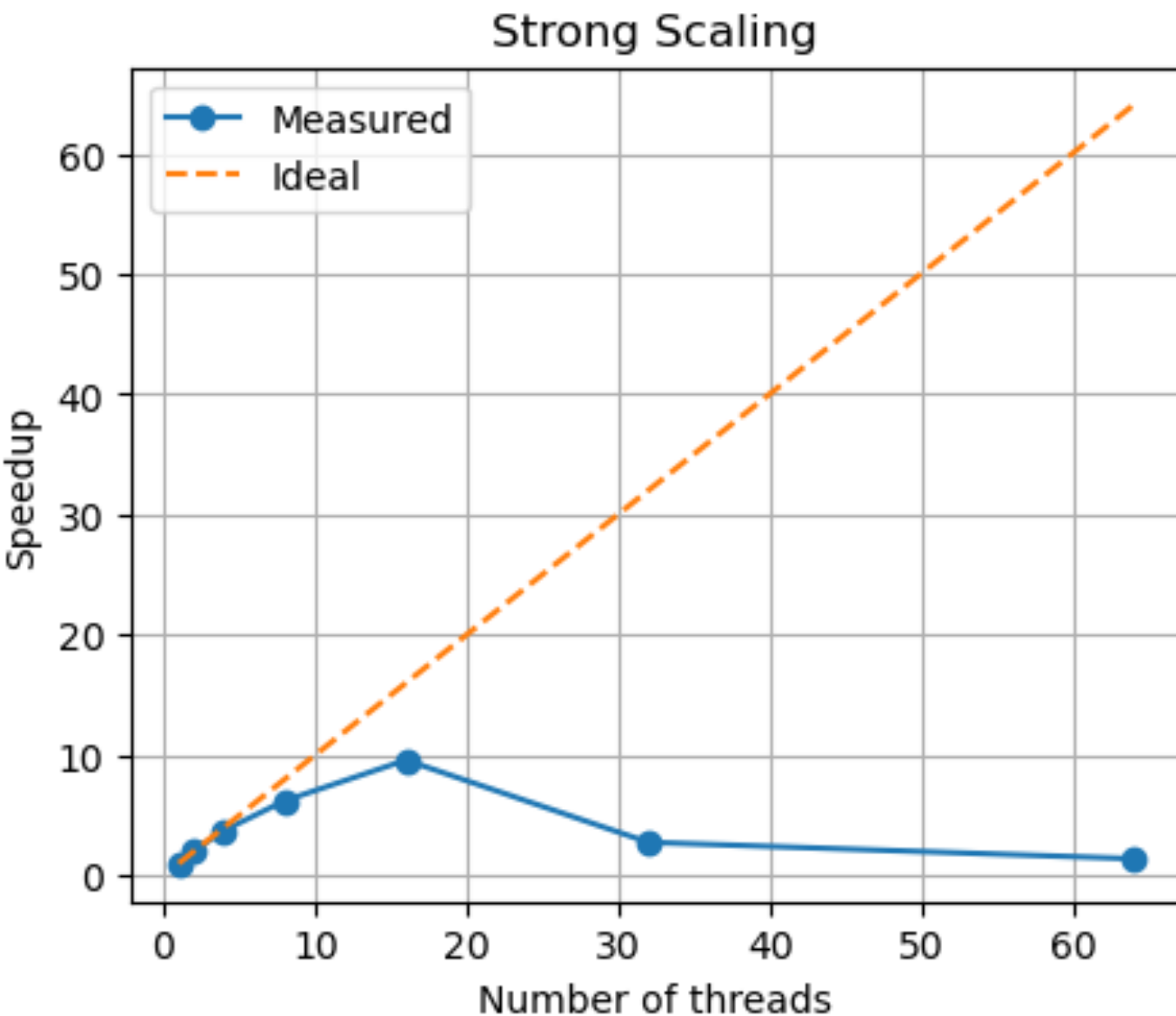
```
#print("N,nthreads,Mem, matrix vector time")
```

```
print(f"{N},{nthreads},{Mem},{dt/nrep}")
```

Hidden Parallelism in Scientific Python: results



Hidden Parallelism in Scientific Python: performance metrics



Speedup: $S(p) = T_1/T(p)$

Ideal speed up : you use p threads the code is p times faster : $T_{\text{ideal}}(p) = T_1/p$

-> $S_{\text{ideal}}(p) = p$

Efficiency: $E(p) = S(p)/p$

Ideal Efficiency -> $E_{\text{ideal}}(p) = 1$

Conclusion: for that problem size I would not use more than 8 threads

Hidden Parallelism in Scientific Python: without restarting kernel

```
#without restarting kernel :
import time
import numpy as np
from threadpoolctl import threadpool_limits
from threadpoolctl import threadpool_info

N = 2000
a = np.random.randn(N, N)
b = np.random.randn(N)

def benchmark(nthreads):
    with threadpool_limits(limits=nthreads):
        nrep = 100
        x = b.copy()
        t1 = time.time()
        for _ in range(nrep):
            x = a @ x

        t2 = (time.time() - t1) / nrep

        print(f"Threads: {nthreads}")
        print(f"matmul : {t2:.6f} s")
    print(threadpool_info())

for nthreads in [1, 2, 4, 8, 16, 32, 64]:
    benchmark(nthreads)
```


Hidden Parallelism in Scientific Python: application to compute_pi

Try with compute_pi (numpy)

-> No speed up . Independently of the number of threads.

Need to try explicit parallelism....

Explicit parallelism with Threads

Idea

- Divide the total work into independent chunks.
- Assign each chunk to a separate thread.
- Combine the partial results at the end.

Advantages

- Simple way to exploit multiple CPU cores.
- Natural approach for independent tasks ("embarrassingly parallel").
- Shared-memory programming model.

Key takeaway: Explicit parallelism gives direct control over threads, but achieving good performance is difficult. Efficient parallel programs require balancing computation, memory access, and threading overhead.

Explicit parallelism

```
import numpy as np
from concurrent.futures import ThreadPoolExecutor

def worker(n):
    x = np.random.random(n)
    y = np.random.random(n)
    return np.sum(x*x + y*y < 1.0)

def compute_pi_numpy_threads(N, nthreads=8):

    chunk = N // nthreads

    with ThreadPoolExecutor(max_workers=nthreads) as pool:
        counts = list(pool.map(worker, [chunk]*nthreads))

    count = sum(counts)

    return 4.0 * count / (chunk * nthreads)
```

Numba & Just-In-Time (JIT) Compilation

- Results : N=100_000_000 / 16 threads

Function	Speed (Million)	Time to 10 ⁻⁷ precision	Time to 10 ⁻⁷ precision
compute_pi_baseline_v0	6.5	~ 372 days	
compute_pi_baseline_v1	7.78	~ 213 days	~ 81 days
compute_pi_baseline_v2	8.39	~ 193 days	~ 43 days
compute_pi_baseline_v3	10.04	~ 167 days	~ 26 days
compute_pi_numpy	81.68	~14 days	
compute_pi_numba	195.36	~6.7 days	
compute_pi_numba_par	3031	~0.46 day	
compute_pi_numpy_threads (64 threads)	123	9.3 days	

GPU acceleration

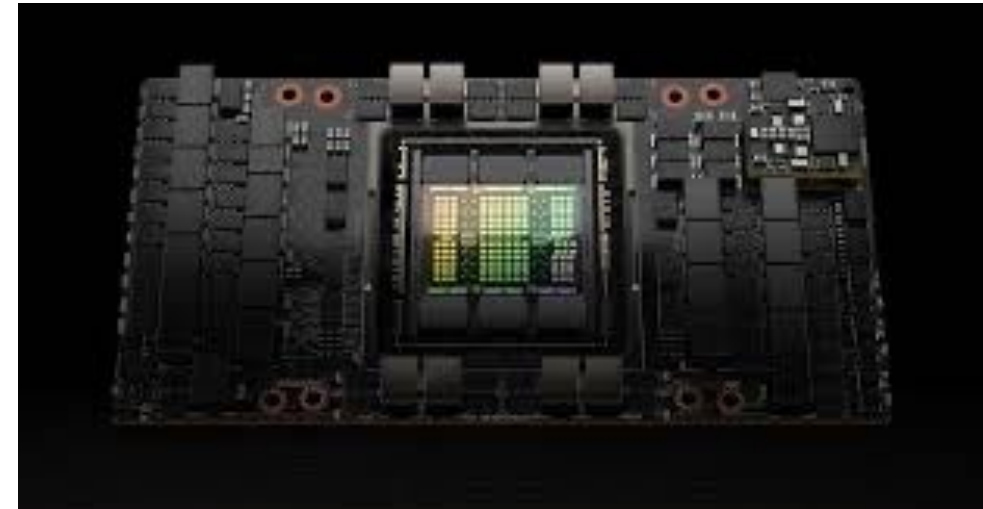
What is Graphical Processing Unit (GPU) ?

CPU node

- **64 CPU cores** available for computation.
- Optimised for sequential tasks
- All cores can access the node's main memory (RAM).
- Different cores can work on the same dataset.

GPU node : A CPU + a GPU

- A CPU node with 64 cores
- 4 GPU card containing thousands of cores
- GPU card optimised for massive parallelism



Why GPUs?

- Many scientific computations perform the same operation on large arrays.
- These operations can be executed simultaneously on thousands of processing units.
- GPUs provide much higher computational throughput than CPUs for suitable workloads.

Our Hardware

- We have access to 4 **NVIDIA H100** or 8 **NVIDIA L40s** accelerators.
- Designed specifically for large-scale scientific computing and AI workloads.
- Among the fastest data-centre GPUs currently available.

Key Idea

GPUs achieve performance through massive parallelism: thousands of operations can be executed concurrently.

GPU programming model

GPU architecture

- A GPU card contains a number of Streaming Multiprocessors (SM). [132 SMs for a H100]
- Each SM contains a number of CUDA cores

Programming strategy

- Split work into many independent tasks.
- Each thread processes a small piece of data.
- Threads execute the same instructions on different data elements.

CUDA (Compute Unified Device Architecture): a proprietary parallel computing platform and programming model created by Nvidia.

A **block** is a group of thread that can cooperate

- Share fast on-chip memory
- Synchronise with one another
- Execute on the same SM

A **CUDA block** is a group of threads. Blocks are scheduled onto SMs which contains the actual CUDA core (hardware).

CUDA hardware executes thread in groups called **warps** (1 warp = 32 threads)

Key Idea

GPU programming extends the same parallel concepts used on CPUs, but at a much larger scale.

Back to numba

Characteristics

- Write custom GPU kernels in Python.
- Explicit control over GPU threads.
- Similar to writing CUDA C++.
- Maximum flexibility and performance.

Use Cases

- Custom numerical algorithms.
- Fine-grained performance tuning.
- Learning GPU programming concepts.

Key Idea

Numba exposes the GPU programming model directly.

Example

```
from numba import njit, prange, cuda
@cuda.jit # tells numba to compile for the GPU. It is called a kernel.
def compute_pi_cuda_kernel(rng_states, results):
    idx = cuda.grid(1)
    if idx < results.size:
        # xoroshiro random numbers
        x = cuda.random.xoroshiro128p_uniform_float32(rng_states, idx)
        y = cuda.random.xoroshiro128p_uniform_float32(rng_states, idx)
        if x*x + y*y < 1.0:
            results[idx] = 1
        else:
            results[idx] = 0

def compute_pi_numba_cuda(Niter):
    from numba.cuda.random import (
        create_xoroshiro128p_states
    )
    threads_per_block = 256
    blocks = (Niter + threads_per_block - 1) // threads_per_block # For N=1_000_000, blocks = 3907

    rng_states = create_xoroshiro128p_states(
        threads_per_block * blocks,
        seed=42
    )

    results = np.zeros(Niter, dtype=np.int32)
    d_results = cuda.to_device(results) # results is transferred to GPU. Memories are separate
    compute_pi_cuda_kernel[blocks, threads_per_block](
        rng_states,
        d_results
    )
    results = d_results.copy_to_host() # transfer data back to CPU memory
    count = results.sum()
    piEst = 4.0 * count / Niter

    return piEst
```

Example

```
from numba import njit, prange, cuda
@cuda.jit # tells numba to compile for the GPU. It is called a kernel.
def compute_pi_cuda_kernel(rng_states, results):
    idx = cuda.grid(1)
    if idx < results.size:
        # xoroshiro random numbers
        x = cuda.random.xoroshiro128p_uniform_float32(rng_states, idx)
        y = cuda.random.xoroshiro128p_uniform_float32(rng_states, idx)
        if x*x + y*y < 1.0:
            results[idx] = 1
        else:
            results[idx] = 0

def compute_pi_numba_cuda(Niter):
    from numba.cuda.random import (
        create_xoroshiro128p_states
    )
    threads_per_block = 256
    blocks = (Niter + threads_per_block - 1) // threads_per_block # For N=1_000_000, blocks = 3907

    rng_states = create_xoroshiro128p_states(
        threads_per_block * blocks,
        seed=42
    )

    results = np.zeros(Niter, dtype=np.int32)
    d_results = cuda.to_device(results) # results is transferred to GPU. Memories are separate
    compute_pi_cuda_kernel[blocks, threads_per_block](
        rng_states,
        d_results
    )
    results = d_results.copy_to_host() # transfer data back to CPU memory
    count = results.sum()
    piEst = 4.0 * count / Niter

    return piEst
```

CuPy: NumPy on the GPU



What is CuPy?

- GPU array library with a NumPy-compatible API.
- Designed as a drop-in replacement for many NumPy programs.
- Uses NVIDIA CUDA GPUs for computation.

Advantages

- Very small code changes.
- Familiar NumPy programming model.
- Many operations automatically execute on the GPU.
- Excellent first step into GPU computing.

Limitations

- CUDA/NVIDIA GPUs only.
- Not every NumPy feature is available.
- Performance depends on minimizing CPU ↔ GPU data transfers.

Key Takeaway

CuPy allows many NumPy programs to run on GPUs simply by replacing `numpy` with `cupy`.

NumPy:

```
import numpy as np
```

```
x = np.random.random(1000000)  
y = np.sin(x)
```

CuPy:

```
import cupy as cp
```

```
x = cp.random.random(1000000)  
y = cp.sin(x)
```

Cupy

```
import cupy as cp

def compute_pi_cupy(Niter) :

    x = cp.random.random(Niter)
    y = cp.random.random(Niter)

    inside = (x*x + y*y) < 1.0

    count = cp.sum(inside)

    piEst = 4.0 * count / Niter

    return float(piEst.get())
```

Results:

Time taken for v0 5000000 : 0.00035262713208794594 s
Estimate for pi is 3.14151668 after 0.00035262713208794594 seconds
Absolute error of the mean is 7.597358979305469e-05
Mean of the Absolute error is 0.0004977707179586232
N/N0 24777568.76570433
speed = 14179.283285419426 million samples per second

Estimation time for 1e-07 precision = 0.10112549784678454 days; Niter target = 1.2388784382852165x10^14

JAX — Compiled Array Programming

What is JAX?

- High-performance numerical computing library developed by Google
- Provides a NumPy-like interface.
- Automatically compiles computations for CPUs and GPUs.



Key Features

- **jax.numpy**: NumPy-compatible array operations.
- **jax.jit**: Just-In-Time compilation.
- **Automatic differentiation** (grad).
- Runs on CPUs, GPUs and TPUs.

Example:

```
import jax

import jax.numpy as jnp

@jax.jit
def f(x):
    return jnp.sin(x) + x**2
```

Why Use JAX?

- Often faster than pure NumPy.
- Automatic optimisation of computations.
- Widely used in machine learning and scientific computing.

Key Takeaway

JAX combines a familiar NumPy interface with automatic compilation and optimisation, enabling high-performance execution on modern hardware.

Jax

```
#with HPC python
from functools import partial
import jax
import jax.numpy as jnp

@partial(jax.jit, static_argnums=(1,))
def compute_pi_jax(key, Niter):

    key1, key2 = jax.random.split(key)

    x = jax.random.uniform(key1, (Niter,))
    y = jax.random.uniform(key2, (Niter,))

    inside = (x*x + y*y) < 1.0

    count = jnp.sum(inside)

    return 4.0 * count / Niter

#warming up
key = jax.random.PRNGKey(42)
compute_pi_jax(key, Niter).block_until_ready()
```

Results:

```
Time taken for v0 5000000 : 1.6295351088047028e-05 s
Estimate for pi is 3.141833782196045 after 1.6295351088047028e-05 seconds
Absolute error of the mean is 0.00024112860625180588
Mean of the Absolute error is 0.00024080276489257812
N/N0 5798597.157991026
speed = 306835.97873921244 million samples per second
```

```
Estimation time for 1e-07 precision = 0.0010936363021714752 days; Niter target = 0.2899298578995513x10^14
```

Final comparison

- Results : N=100_000_000 / 16 threads

Function	Speed (Million precision)	Time to 10 ⁻⁷ precision	Time to 10 ⁻⁷ precision
compute_pi_baseline_v0	6.5	~ 372 days	
compute_pi_baseline_v1	7.78	~ 213 days	~ 81 days
compute_pi_baseline_v2	8.39	~ 193 days	~ 43 days
compute_pi_baseline_v3	10.04	~ 167 days	~ 26 days
compute_pi_numpy	81.68	~14 days	
compute_pi_numba	195.36	~6.7 days	
compute_pi_numba_par	3031	~0.46 day	
compute_pi_numpy_threads (64 threads)	123	9.3 days	
compute_pi_numba_cuda	?	?	
compute_pi_cupy	14179	2.4 hours	
compute_pi_jax	306835	86 sec ?	

Conclusion: From 1 year to a minute !

Conclusion

Several techniques to optimise your code;

- Profiling and identifying bottlenecks.
- NumPy vectorisation.
- Just-In-Time compilation with Numba.
- Shared-memory parallelism and multithreading.
- GPU computing with CUDA, CuPy and JAX.

Why Does Performance Matter?

- Faster code means less waiting and more productivity.
- Interactive workflows become more enjoyable.
- Larger and more ambitious simulations become feasible.
- Computational resources are used more efficiently.computing.

Final words

Think Bigger

A 10× speedup does not just save time:

- It may allow you to run 10× more simulations.
- It may allow you to increase the problem size dramatically.
- It may make previously infeasible computations possible.